

RED & WHITE SKILL EDUCATION

DEEP LEARNING

PR 1

Project Definition

Topics: SLP · MLP · Data Scaling · Activation Functions · Early Stopping · Dropout · Regularization

Total Marks: 10

Field	Detail
Institute	Red & White Skill Education
Subject	Deep Learning
Project	PR 1
Total Marks	10
Topics	SLP · MLP · Data Scaling · Activation Functions · Early Stopping · Dropout Layers · Regularization
Dataset	Breast Cancer Wisconsin (Diagnostic) Dataset — sklearn / UCI ML Repository
Framework	TensorFlow 2.x / Keras (Python 3.x, Jupyter Notebook)

§ 1 · Dataset

This project uses the Breast Cancer Wisconsin (Diagnostic) Dataset — a classic, well-understood binary classification dataset that is built directly into scikit-learn and also available on the UCI ML Repository. Its 30 numeric features and clean structure make it ideal for building and comparing neural network architectures without the distraction of heavy data cleaning. Students will progress from a Single-Layer Perceptron baseline all the way to a regularised, dropout-equipped MLP, measuring the impact of each deep learning technique along the way.

Field	Detail
Dataset Name	Breast Cancer Wisconsin (Diagnostic)
Source	<code>sklearn.datasets.load_breast_cancer()</code> — also available at UCI ML Repository
UCI URL	https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
Loading	from sklearn.datasets import load_breast_cancer → data = load_breast_cancer() Or Use from given UCI URL
Total Samples	569 samples (212 Malignant · 357 Benign)
Features	30 numeric features — mean, standard error, and worst values of 10 cell-nucleus measurements
Target	Binary: 0 = Malignant · 1 = Benign
Missing Values	None — dataset is completely clean
License	Public Domain — UCI ML Repository

Key feature groups:

Column Name	Data Type	Description
radius_mean / _se / _worst	<i>Decimal</i>	Mean, std-error, and worst radius of cell nucleus across all cells in the image
texture_mean / _se / _worst	<i>Decimal</i>	Standard deviation of grey-scale values
perimeter_mean / _se / _worst	<i>Decimal</i>	Perimeter of the nucleus
area_mean / _se / _worst	<i>Decimal</i>	Area of the nucleus
smoothness_mean / _se / _worst	<i>Decimal</i>	Local variation in radius lengths

compactness_mean / _se / _worst	Decimal	Perimeter ² / area – 1.0
concavity_mean / _se / _worst	Decimal	Severity of concave portions of the contour
concave points_mean / _se / _worst	Decimal	Number of concave portions of the contour
symmetry_mean / _se / _worst	Decimal	Symmetry of the nucleus
fractal_dimension_mean / _se / _worst	Decimal	'Coastline approximation' – 1 (complexity measure)
target	Integer (0/1)	0 = Malignant · 1 = Benign (class label)

Note: Load via sklearn — no CSV download required. Call `load_breast_cancer(as_frame=True)` to get a pandas DataFrame directly. Apply StandardScaler to all 30 features before training any model. Use `train_test_split` with `test_size=0.2`, `random_state=42`, `stratify=y` to preserve class balance.

§ 2 · Project Tasks — What to Do

Complete ALL tasks in a single Jupyter Notebook named DL_PR1.ipynb. Structure the notebook with Markdown headings that match the task titles below. Each task builds directly on the previous — do not skip tasks or re-order them. Your goal is to understand WHY each technique is applied, not just how to write the code.

Task 1: Data Loading, EDA & Preprocessing

#	Action	Details / Configuration
1.1	Load the dataset	<code>from sklearn.datasets import load_breast_cancer</code> . Call <code>load_breast_cancer(as_frame=True)</code> . Assign <code>X = data.data</code> and <code>y = data.target</code> . Print <code>X.shape</code> (should be <code>569×30</code>), <code>y.value_counts()</code> , and <code>X.describe()</code> .
1.2	Class distribution plot	<code>sns.countplot(x=y)</code> . Title: 'Target Class Distribution — Malignant (0) vs Benign (1)'. Add counts as annotations above each bar. Add a Markdown cell: note the mild class imbalance (212 vs 357) and whether it warrants special handling for a binary classification task at this dataset size.
1.3	Feature correlation heatmap	Compute <code>X.corr()</code> . Plot <code>sns.heatmap</code> with <code>annot=False</code> (<code>30×30</code> is too large for annotations), <code>cmap='coolwarm'</code> , <code>linewidths=0.3</code> . Title: 'Feature Correlation Heatmap'. In a Markdown cell: identify 2–3 highly correlated feature pairs (e.g., <code>radius_mean</code> and <code>perimeter_mean</code>). Explain why high inter-feature correlation is not a problem for neural networks the way it is for linear models.
1.4	Train / test split	<code>from sklearn.model_selection import train_test_split</code> . <code>X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)</code> . Print shapes of all four arrays. Confirm the stratify parameter preserved the ~63/37 class ratio in both splits.
1.5	Apply StandardScaler	<code>from sklearn.preprocessing import StandardScaler</code> . Fit scaler on <code>X_train</code> only (<code>scaler.fit(X_train)</code>). Transform both <code>X_train</code> and <code>X_test</code> (never fit on <code>X_test</code> — explain why in a Markdown cell). Store scaled arrays as <code>X_train_sc</code> and <code>X_test_sc</code> . Verify mean ≈ 0 and std ≈ 1 on <code>X_train_sc</code> using <code>.mean(axis=0).round(2)</code> .
1.6	WHY scaling for neural networks	Add a Markdown cell: explain that gradient-based optimisers (SGD, Adam) converge faster and more stably when input features are on the same scale. Without scaling, features with large ranges (e.g., <code>area_mean</code> up to ~2500) dominate early gradient updates, while small-range features (e.g., <code>fractal_dimension_mean</code> ~0.05–0.10) contribute negligibly — causing slow, biased learning. <code>StandardScaler</code> removes this disparity.

Task 2: Single-Layer Perceptron (SLP) — Baseline Model

#	Action	Details / Configuration
2.1	Build the SLP	<code>import tensorflow as tf</code> from <code>tensorflow</code> import <code>keras</code> . <code>model_slp = keras.Sequential([keras.layers.Dense(1, activation='sigmoid', input_shape=(30,))])</code> . This is a true SLP: one neuron, one weight per feature, one bias. No hidden layers.

2.2	Compile the SLP	<code>model_slp.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])</code> . Print <code>model_slp.summary()</code> . Confirm total parameters = 31 (30 weights + 1 bias). Add a Markdown cell: explain why <code>binary_crossentropy</code> is the correct loss function for binary classification (it measures the log-likelihood of the predicted probability vs the true label, unlike MSE which is designed for regression).
2.3	Train the SLP	<code>history_slp = model_slp.fit(X_train_sc, y_train, epochs=50, batch_size=32, validation_split=0.1, verbose=1)</code> . Store the returned History object — you will plot it.
2.4	Plot SLP training curves	Create a 1×2 subplot: (left) Training vs Validation Loss over epochs; (right) Training vs Validation Accuracy over epochs. Use Red #CC0000 for training curves, Charcoal #2C2C2C for validation curves. Add a legend and descriptive title. In a Markdown cell: describe what the curves reveal — does the SLP converge smoothly? Is there a gap between training and validation performance?
2.5	Evaluate the SLP	<code>slp_loss, slp_acc = model_slp.evaluate(X_test_sc, y_test, verbose=0)</code> . Print test accuracy. Generate: <code>y_pred_slp = (model_slp.predict(X_test_sc) > 0.5).astype(int)</code> . Print <code>sklearn.classification_report(y_test, y_pred_slp)</code> . Plot confusion matrix with <code>sns.heatmap</code> . Record test accuracy in a results comparison table you will build in Task 7.
2.6	SLP limitations — Markdown	Add a Markdown cell: explain the fundamental limitation of an SLP — it can only learn linearly separable decision boundaries (one hyperplane through 30-dimensional space). Breast cancer classification is unlikely to be perfectly linearly separable, so the SLP sets a realistic but improvable baseline. This motivates the MLP in Task 3.

Task 3: Multi-Layer Perceptron (MLP) — Activation Functions

#	Action	Details / Configuration
3.1	Build the MLP (ReLU)	<code>model_mlp = keras.Sequential([keras.layers.Dense(64, activation='relu', input_shape=(30,)), keras.layers.Dense(32, activation='relu'), keras.layers.Dense(1, activation='sigmoid')])</code> . Print <code>model_mlp.summary()</code> . Count total parameters manually ($30 \times 64 + 64 \times 32 + 32 \times 1 + 1$) and verify against the summary — this reinforces understanding of weight matrix dimensions.
3.2	WHY hidden layers — Markdown	Add a Markdown cell: explain that each Dense layer learns a non-linear transformation of its inputs. Stacking two hidden layers allows the network to compose two levels of non-linear feature interactions — a single hidden layer can approximate any function in theory (Universal Approximation Theorem), but two layers often converge faster in practice for tabular medical data. More layers → deeper feature hierarchy.
3.3	Compile and train MLP (ReLU)	<code>model_mlp.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])</code> . <code>history_mlp = model_mlp.fit(X_train_sc, y_train, epochs=100, batch_size=32, validation_split=0.1, verbose=1)</code> .
3.4	Compare activation functions	Build two additional identical MLP architectures (same layer sizes), substituting: (a) <code>activation='tanh'</code> throughout both hidden layers; (b) <code>activation='sigmoid'</code> throughout both hidden layers. Train both for 100 epochs. Plot a 3-panel figure (1×3): training validation accuracy over epochs for ReLU / Tanh / Sigmoid side-by-side. Title: 'MLP Training Accuracy — ReLU vs Tanh vs Sigmoid (Hidden Layers)'.

3.5	Activation functions — Markdown	Add a Markdown cell comparing the three activation functions used in hidden layers: (1) ReLU: output = $\max(0, x)$; zero gradient for negative inputs (dying ReLU risk); no vanishing gradient for positive inputs; fastest convergence on most tabular tasks. (2) Tanh: output in $(-1, 1)$; zero-centred unlike Sigmoid; can suffer vanishing gradients in deep networks. (3) Sigmoid in hidden layers: output in $(0, 1)$; not zero-centred; severe vanishing gradient problem in deep nets — use only in the output layer for binary classification. State which activation performed best on this dataset and why.
3.6	Evaluate best MLP	Evaluate the best-performing activation variant on <code>X_test_sc</code> . Record test accuracy. Print <code>classification_report</code> . Plot confusion matrix. Compare with SLP results from Task 2.5 in the results table.

Task 4: Early Stopping

#	Action	Details / Configuration
4.1	Build MLP for Early Stopping experiment	Build a fresh MLP: <code>Dense(128, relu) → Dense(64, relu) → Dense(1, sigmoid)</code> . Deliberately train for a large number of epochs (<code>epochs=300</code>) to create conditions where overfitting would naturally occur without intervention. Compile with <code>adam</code> and <code>binary_crossentropy</code> .
4.2	Add EarlyStopping callback	<pre>from tensorflow.keras.callbacks import EarlyStopping. es = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True, verbose=1). Pass callbacks=[es] to model.fit(). history_es = model_es.fit(X_train_sc, y_train, epochs=300, batch_size=32, validation_split=0.1, callbacks=[es], verbose=1).</pre>
4.3	WHY Early Stopping — Markdown	Add a Markdown cell explaining all four EarlyStopping parameters used: (1) <code>monitor='val_loss'</code> — watch validation loss, not training loss, because training loss always decreases; only validation loss reveals true generalisation. (2) <code>patience=15</code> — allow 15 consecutive epochs of no improvement before stopping; prevents halting on a temporary plateau. (3) <code>restore_best_weights=True</code> — rewinds model weights to the epoch with the lowest <code>val_loss</code> after stopping; without this, the final model weights are from the last epoch (post-overfit), not the best epoch. (4) <code>verbose=1</code> — prints the epoch at which training stopped.
4.4	Plot Early Stopping curves	Plot training vs validation loss over epochs. Draw a vertical dashed red line at the epoch where training actually stopped (<code>len(history_es.history['loss'])</code>). Annotate the line: 'Training stopped — best weights restored'. Title: 'Early Stopping — Val Loss with Best Epoch Marked'.
4.5	Compare: with vs without Early Stopping	Train a second identical MLP (same architecture) for the full 300 epochs without any callback. Plot the validation loss curves of both models on the same axes (one colour each, with legend). In a Markdown cell: identify the epoch range where the no-callback model begins overfitting (<code>val_loss</code> starts rising). Confirm that the EarlyStopping model's final <code>val_loss</code> is lower. Evaluate both on the test set and compare test accuracy.

Task 5: Dropout Layers

#	Action	Details / Configuration
---	--------	-------------------------

5.1	Build MLP with Dropout	<code>model_drop = keras.Sequential([keras.layers.Dense(128, activation='relu', input_shape=(30,)), keras.layers.Dropout(0.3), keras.layers.Dense(64, activation='relu'), keras.layers.Dropout(0.3), keras.layers.Dense(1, activation='sigmoid')])</code> . Print <code>model_drop.summary()</code> . Note that Dropout layers have zero trainable parameters — they only randomly zero activations during training.
5.2	WHY Dropout — Markdown	Add a Markdown cell: Dropout(rate=0.3) randomly sets 30% of neuron activations to zero during each training batch. This forces the network to learn redundant representations — no single neuron can be relied upon, so the network distributes knowledge across many neurons. At inference time (<code>model.predict()</code>), Dropout is automatically disabled and all neurons contribute (scaled by the keep probability). Dropout is equivalent to training an ensemble of 2^n thinned networks and averaging their predictions, which reduces variance and improves generalisation.
5.3	Train with Dropout + Early Stopping	Compile <code>model_drop</code> with adam and binary_crossentropy. Add <code>EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True)</code> . <code>history_drop = model_drop.fit(X_train_sc, y_train, epochs=300, batch_size=32, validation_split=0.1, callbacks=[es], verbose=1)</code> .
5.4	Effect of Dropout rate	Build two more models with identical architecture but Dropout rates of 0.1 and 0.5. Train each for 200 epochs with Early Stopping. Plot validation accuracy of all three dropout rates (0.1, 0.3, 0.5) on the same axes over epochs. Title: 'Dropout Rate Comparison — Validation Accuracy'. In a Markdown cell: explain the trade-off — too low a rate (0.1) gives insufficient regularisation; too high a rate (0.5) removes so many activations per batch that the network underfits and learning slows significantly.
5.5	Evaluate Dropout model	Evaluate the best Dropout configuration on <code>X_test_sc</code> . Record test accuracy and add to the results table. Compare training accuracy vs validation accuracy with and without Dropout — the gap should be smaller with Dropout (less overfitting).

Task 6: Regularization (L1, L2, L1-L2)

#	Action	Details / Configuration
6.1	Build MLP with L2 Regularization	<code>from tensorflow.keras import regularizers</code> . <code>model_l2 = keras.Sequential([keras.layers.Dense(128, activation='relu', input_shape=(30,)), kernel_regularizer=regularizers.l2(0.001)), keras.layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001)), keras.layers.Dense(1, activation='sigmoid')])</code> . Compile and train with EarlyStopping (patience=20). Store <code>history_l2</code> .
6.2	WHY L2 Regularization — Markdown	Add a Markdown cell: L2 regularization adds $\lambda \times \sum w^2$ to the loss function at each training step. This penalises large weight values, pushing all weights toward zero but never exactly to zero. The effect is a smoother decision boundary — the model relies on many small weights rather than a few dominant ones, which reduces variance (overfitting). $\lambda=0.001$ is a moderate penalty; too large a λ causes underfitting by over-shrinking all weights.
6.3	Build MLP with L1 Regularization	<code>model_l1 = same architecture using regularizers.l1(0.001)</code> . Train with EarlyStopping. Store <code>history_l1</code> . In a Markdown cell: explain how L1 ($\lambda \times \sum w $) differs from L2 — L1 produces sparse weight vectors (many weights become exactly zero), effectively performing feature selection.

		On a 30-feature dataset like Breast Cancer, L1 may zero out weights for weakly predictive features.
6.4	Build MLP with L1-L2 (ElasticNet)	model_l1l2 = same architecture using regularizers.l1_l2(l1=0.0001, l2=0.001). Train with EarlyStopping. In a Markdown cell: explain that ElasticNet combines both penalties — L2 handles correlated features (many of which exist in this dataset: radius, perimeter, area are highly correlated), while L1 drives truly uninformative weights to zero.
6.5	Regularization comparison plot	Plot a 1×3 figure: training vs validation loss for L1 (left) / L2 (centre) / L1-L2 (right) — all on the same y-axis scale for fair comparison. Title: 'Regularization Comparison — Training vs Validation Loss'. In a Markdown cell: which regularizer showed the smallest train–validation loss gap? Which reached the best validation accuracy? Does the answer match theory?
6.6	Evaluate all three regularized models	Evaluate L1, L2, and L1-L2 models on X_test_sc. Record all test accuracies in the results table. Plot a grouped bar chart comparing test accuracy across: No Regularization (Task 3 best MLP), L1, L2, L1-L2. Use Red #CC0000 as the primary bar colour.

Task 7: Final Comparison, Business Insights & Notebook Quality

#	Action	Details / Configuration
7.1	Best model: combine Dropout + L2 + Early Stopping	Build a final model combining the best techniques from Tasks 4–6: Dense(128, relu, L2) → Dropout(0.3) → Dense(64, relu, L2) → Dropout(0.3) → Dense(1, sigmoid). Train with EarlyStopping(patience=20, restore_best_weights=True). This is your production-quality model.
7.2	Full results comparison table	Build a pandas DataFrame with one row per model and columns: Model, Architecture, Regularization, Dropout Rate, Early Stopping, Test Accuracy, Test Precision, Test Recall, Test F1-Score. Include: SLP (Task 2), MLP-ReLU (Task 3), MLP + Early Stopping (Task 4), MLP + Dropout-best (Task 5), MLP + L2 (Task 6), Final Combined Model (Task 7.1). Display with df.style.highlight_max(subset=['Test Accuracy', 'Test F1-Score'], color='#ffcccc').
7.3	Business insight report — Markdown	Write a structured Markdown section titled 'Clinical Insight for Medical Diagnosis Support'. Cover: (a) Which model would you recommend for deployment in a clinical decision-support tool — and why? Consider precision vs recall trade-off (in cancer diagnosis, false negatives — missing a malignant case — are far more costly than false positives). (b) What threshold would you set for classification (default is 0.5 — should it be lower for this use case)? (c) Which deep learning technique (Early Stopping / Dropout / Regularization) had the largest positive impact on generalisation for this dataset?
7.4	Notebook quality standards	Every task must have an opening Markdown cell: task title + objective (1 sentence) + expected output. All plots: descriptive title, labelled axes, legend. sns.set_style('whitegrid') at the top of the notebook. No dead code cells. Restart kernel → Kernel > Restart & Run All → zero errors before submission. Export as HTML: File > Download as > HTML (.html). Upload both .ipynb and .html to GitHub.
7.5	requirements.txt	Create requirements.txt in the repo root: tensorflow>=2.12.0, scikit-learn>=1.4.0, pandas, numpy, matplotlib, seaborn. Pin major versions. This ensures any reviewer can reproduce your environment without dependency conflicts.

§ 3 · Transparent Marking Scheme (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb)	5 Marks	50%	Completed notebook — all 7 tasks, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + .html + plots + README + video link
TOTAL	10 Marks	100%	

NOTE:

- Each **late submission** will **decrease the marks by 1 per one day**.
- For example, if you are 2 days late submitting the project, your 2 marks will be cut.
- So respect the given deadline, and according to that, submit the project within the given timeline.

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	0.5	Dataset loaded via sklearn; X (569×30) and y confirmed; class distribution plot; correlation heatmap; stratified train/test split (80/20); StandardScaler fitted on train only; Markdown explaining WHY scaling is required for gradient-based optimisers	Loaded and scaled, minor EDA gaps	Loaded and split, no scaling explanation	CSV connected, no preprocessing	Not done
A2	1.0	SLP built correctly (1 neuron, sigmoid output, 31 params); compiled with binary_crossentropy; trained 50 epochs; training curves plotted (loss + accuracy);	SLP built and evaluated, partial analysis	SLP built, basic accuracy only	SLP attempted with errors	No SLP

		confusion matrix and classification_report on test set; SLP limitation explained in Markdown; results recorded in comparison table				
A3	1.5	MLP (64→32→1) built with ReLU; parameter count verified; trained 100 epochs; all 3 activation function variants (ReLU / Tanh / Sigmoid hidden) trained and compared in 3-panel plot; activation function properties explained in Markdown (vanishing gradient, zero-centring, convergence speed); best variant evaluated on test set and recorded	MLP with 2 activation variants, partial comparison	MLP with ReLU only, no comparison	MLP attempted, architecture errors	No MLP
A4	1.0	Early Stopping callback: all 4 parameters explained (monitor, patience, restore_best_weights, verbose); training curves with stopping epoch marked as vertical dashed line; side-by-side comparison (with vs without Early Stopping) on same axes; both models evaluated on test set; val_loss improvement quantified in Markdown	EarlyStopping applied and plotted, partial explanation	EarlyStopping applied, no comparison	EarlyStopping attempted, parameter errors	No Early Stopping
A5	1.0	Dropout: rate comparison (0.1, 0.3, 0.5) with validation accuracy plot; effect explained (redundant representations, inference scaling); L1 / L2 / L1-L2 all implemented and compared; regularization comparison 3-panel loss plot; final combined model (Dropout + L2 + EarlyStopping); full results table with 6+ models; business insight report with precision/recall trade-off and threshold discussion; Restart & Run All clean	Dropout + one regularizer + partial comparison;	Dropout OR regularization only; basic results table	One technique partially implemented	None

Subtotal A: 0.5 + 1.0 + 1.5 + 1.0 + 1.0 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your Jupyter Notebook screen simultaneously. Explain concepts verbally throughout — do not just execute cells without speaking.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	1.0	Clear audio and video; face visible via PiP throughout; notebook cell outputs, model summaries, and all plots clearly visible on screen; all 7 tasks demonstrated sequentially	Clear audio, face mostly visible	Audio unclear; face partially visible	Very poor quality, hard to follow	No video
B2	1.5	Verbally explains all 7 concepts: (1) WHY StandardScaler is applied before gradient descent; (2) SLP vs MLP — what hidden layers add and why; (3) ReLU vs Tanh vs Sigmoid in hidden layers — vanishing gradients, zero-centring; (4) Early Stopping parameters — especially <code>restore_best_weights</code> and why <code>val_loss</code> is monitored, not <code>train_loss</code> ; (5) HOW Dropout works during training vs inference — the ensemble interpretation; (6) Difference between L1 (sparsity) and L2 (weight shrinkage) — and when to prefer each; (7) Clinical interpretation — precision vs recall trade-off for cancer diagnosis	Explains 5–6 concepts with reasoning	Demonstrates steps without explaining why	Reads code from screen, no conceptual analysis	No verbal explanation

		and threshold reasoning				
B3	0.5	5–10 min duration; covers all 7 tasks; shows model.summary() for SLP and MLP; shows all training curve plots live; shows results comparison table; demonstrates the effect of Early Stopping (stopping epoch visible)	4–5 min; covers most tasks	2–4 min; misses key technique steps	Under 2 min or very incomplete	Not submitted

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Video Requirements

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Jupyter Notebook — show entire screen
Duration	5 to 10 minutes
Naming convention	DL_PR1_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (2 Marks)

Create a public GitHub repository specifically for this project.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	0.5	Repo contains: DL_PR1.ipynb + DL_PR1.html + /plots folder with key figures (training curves, activation comparison, dropout rate comparison, regularization comparison, results	Key files present, a few missing	Only .ipynb uploaded, no plots	Repo exists but empty or broken	No GitHub repo

		bar chart) clearly named + requirements.txt				
C2	1.0	README contains: project title + dataset source (sklearn) + all 7 topics covered with 1-line descriptions + /plots screenshots with captions + full results table (copy-pasted from notebook) + video link + tools used (TensorFlow, sklearn, seaborn) + requirements listed	Title + dataset + partial topic docs	Title and dataset only, no topic docs	Blank README or single line	No README
C3	0.5	3+ meaningful commits with descriptive messages, e.g.: 'Add SLP baseline — binary_crossentropy, 50 epochs, confusion matrix'; 'Build MLP — ReLU vs Tanh vs Sigmoid activation comparison'; 'Add Early Stopping — patience=15, restore_best_weights, stopping epoch plot'	2–3 adequate commits with basic messages	1 bulk commit with all files at once	Commit messages unclear or auto-generated	No commits or single initial commit

Subtotal C: $0.5 + 1.0 + 0.5 = 2.0$ Marks

§ 4 · Submission Checklist

Tick every item below before submitting. All items are required for full marks.

- ☐ Dataset loaded via `sklearn.datasets.load_breast_cancer(as_frame=True)` — no CSV download required
- ☐ `X.shape` confirmed as (569, 30); `y.value_counts()` shows 212 Malignant (0) and 357 Benign (1)
- ☐ Class distribution countplot with bar annotations generated
- ☐ Correlation heatmap (30×30) generated with `coolwarm` colormap
- ☐ Stratified train/test split: `test_size=0.2`, `random_state=42`, `stratify=y`
- ☐ `StandardScaler` fitted on `X_train` only — `X_train_sc` and `X_test_sc` created
- ☐ Markdown cell: explanation of WHY scaling is required before gradient-based optimisation
- ☐ SLP built: `Dense(1, sigmoid, input_shape=(30,))` — exactly 31 parameters confirmed in `model.summary()`
- ☐ SLP compiled with `binary_crossentropy` loss and `adam` optimiser
- ☐ SLP trained for 50 epochs with `validation_split=0.1`
- ☐ SLP training curves (loss + accuracy) plotted as 1×2 subplot
- ☐ SLP evaluated on test set: `classification_report` and confusion matrix heatmap generated
- ☐ Markdown: SLP linear decision boundary limitation explained — motivates MLP
- ☐ SLP test accuracy recorded in results comparison table

- ☐ MLP built: Dense(64, relu) → Dense(32, relu) → Dense(1, sigmoid) — parameter count verified manually
- ☐ Markdown: Universal Approximation Theorem and WHY hidden layers improve representation explained
- ☐ All 3 activation variants trained: ReLU / Tanh / Sigmoid in hidden layers (100 epochs each)
- ☐ 3-panel activation comparison plot generated (training validation accuracy per variant)
- ☐ Markdown: ReLU vs Tanh vs Sigmoid properties — vanishing gradient, zero-centring, convergence speed
- ☐ Best MLP variant evaluated on test set and recorded in results table
- ☐ MLP (128→64→1) built for Early Stopping experiment
- ☐ EarlyStopping callback: monitor='val_loss', patience=15, restore_best_weights=True, verbose=1
- ☐ Markdown: all 4 EarlyStopping parameters explained — especially restore_best_weights rationale
- ☐ Training curve plotted with vertical dashed red line at stopping epoch
- ☐ Side-by-side comparison: Early Stopping vs no callback on same axes
- ☐ Both models evaluated on test set; val_loss improvement quantified in Markdown
- ☐ Dropout MLP built: Dense(128) → Dropout(0.3) → Dense(64) → Dropout(0.3) → Dense(1)
- ☐ Markdown: Dropout mechanism during training vs inference (ensemble interpretation) explained
- ☐ Dropout rate comparison: 0.1 / 0.3 / 0.5 — validation accuracy over epochs on same axes
- ☐ Markdown: underfitting risk at high dropout rates vs insufficient regularisation at low rates
- ☐ Best Dropout configuration evaluated on test set and recorded
- ☐ L2 regularization MLP built: kernel_regularizer=regularizers.l2(0.001) on both hidden layers
- ☐ Markdown: L2 penalty ($\lambda \times \Sigma w^2$) — weight shrinkage, smoother boundary, λ trade-off explained
- ☐ L1 regularization MLP built: kernel_regularizer=regularizers.l1(0.001)
- ☐ Markdown: L1 sparsity (many weights → exactly zero) vs L2 shrinkage — feature selection effect
- ☐ L1-L2 (ElasticNet) MLP built: regularizers.l1_l2(l1=0.0001, l2=0.001)
- ☐ 3-panel regularization comparison plot: training vs validation loss for L1 / L2 / L1-L2
- ☐ Grouped bar chart: test accuracy across No Regularization / L1 / L2 / L1-L2
- ☐ Final combined model: Dense(128, L2) → Dropout(0.3) → Dense(64, L2) → Dropout(0.3) → Dense(1) with EarlyStopping
- ☐ Full results table: 6+ models × (Architecture, Regularization, Dropout, EarlyStopping, Accuracy, Precision, Recall, F1)
- ☐ Business insight report: deployment recommendation with clinical reasoning
- ☐ Every task has opening Markdown cell: title + objective + expected output
- ☐ All plots: descriptive title, labelled axes, legend, sns.set_style('whitegrid') applied globally
- ☐ Kernel restarted — Restart & Run All executed with zero errors before submission
- ☐ Notebook exported as HTML — both DL_PR1.ipynb and DL_PR1.html present
- ☐ requirements.txt committed with TensorFlow, sklearn, pandas, numpy, matplotlib, seaborn + version pins
- ☐ Video: face (PiP) + Jupyter screen, 5–10 min, MP4/MOV, named DL_PR1_YourName_GRID.mp4
- ☐ Video covers all 7 concepts verbally: scaling, SLP limits, activation functions, Early Stopping params, Dropout mechanism, L1 vs L2, clinical trade-off
- ☐ GitHub: public repository with DL_PR1.ipynb + DL_PR1.html + /plots folder + requirements.txt
- ☐ README.md: dataset source + all 7 topics + plots with captions + full results table + video link + tools used
- ☐ 3+ meaningful Git commits with descriptive messages referencing specific techniques
- ☐ Video link pasted in GitHub README.md under the Video section

© Red & White Skill Education. All Rights Reserved.